Configuration Comparison Automation with Claude: Research and Content Plan

Network and Infrastructure Automation · 3 days (8 hours per day) · Intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.

- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References

1. Where this course came from

Client input.

> From the client's use-case list (Yusuf), item 8, Configuration Comparison Automation: "Planning automation and comparing configuration of 2 different platforms, may be cloud-cloud, physical to cloud or cloud to physical." The list closes with: "Our Aim is to do automation using Claude for all these content."

Load balancer migrations fail quietly. A configuration rebuilt on a new platform can look complete and still differ in a persistence timeout, a monitor interval or a pool's failover design, and the difference only shows up after cut-over. The client asked for automation that plans these migrations and compares configurations across platforms in all three directions: cloud to cloud, physical to cloud and cloud to physical.

Vendor tools help within one ecosystem. NetScaler can diff two ns.conf files [2] and migrate applications with StyleBooks [3], and F5 offers ACC and Journeys for AS3 and platform migrations [6] [7]. None of them compares an F5 with a NetScaler, an ALB [4] or an Application Gateway [5], and the dialects differ in objects, defaults and units, for example F5's 180-second source-address persistence default [18] versus NetScaler's lbvserver timeout default of 2 [16].

The course teaches a pipeline that closes that gap without giving an agent write access to anything: read-only collectors, a canonical Pydantic model with provenance, a deterministic diff engine, and Claude used for planning and explanation through scoped tools and structured output [8][12][13]. Results go through GitHub pull requests and ServiceNow changes [26][27], so every difference is verified, explained and approved before the change window opens.

2. How the outline was shaped

Decision	Rationale
Treat 'planning automation' and 'comparing configuration' as two halves: a Claude planning agent and a deterministic diff engine.	The client input names both; planning benefits from language reasoning, while the comparison itself must be repeatable and testable.
Normalize every platform into one canonical model before comparing.	Different dialects, defaults and units make raw diffs meaningless [1][2][16][18]; a canonical model makes all three directions use the same engine.
Claude never computes the diff and has no write tools.	Determinism and safety: the engine is tested with seeded mismatches, and MCP guidance calls for human control over tool use [14].
Use F5 to NetScaler as the main physical-to-cloud scenario, with ALB and Application Gateway as cloud references and Terraform state for cloud-side resources.	F5 and NetScaler are the platforms named across the client's use cases; Terraform is how cloud load balancers are commonly defined [9] [24].
Include Radware Alteon, 'Anaya LB' and Cisco APIC collectors, using fixtures where lab access is not available.	The client's estate includes these platforms; recorded fixtures keep the lab affordable and reproducible [22][23].
Gate results through GitHub pull requests and ServiceNow changes.	These are the client's existing review and change systems, and both have documented API and protection features [26][27][31].

3. Audience and prerequisites

Role	What they need	Starting point
Load balancer and ADC engineer	Prove a migrated configuration matches before cut-over.	Expert in one vendor's CLI, some Python, little cloud load balancer experience.
Cloud migration or platform engineer	Map appliance features to ALB, App Gateway or cloud NetScaler and detect drift from Terraform.	Comfortable with Terraform and cloud consoles, less with appliance configuration.
Network operations team member	Readable drift summaries and remediation steps.	Uses ServiceNow daily, limited coding.
Infrastructure automation lead	A governed, testable pattern for using Claude in change-related automation.	Python, Git and CI experienced.

Prerequisites from the outline

- Working knowledge of load balancers or ADCs (F5, NetScaler, Radware or similar) and basic cloud networking
- Comfort with the command line, REST APIs and JSON
- Basic Python (reading and editing scripts)
- Familiarity with Git; prior use of Claude or an AI coding assistant is helpful, not required

Checks before enrolment

- Can you explain virtual server, pool, monitor, persistence and SSL profile, and name the equivalent objects on a second platform?
- Can you call a REST API with curl and read JSON?
- Can you read and modify a Python script and run pytest?
- Can you run terraform init and terraform plan?
- Can you branch, commit and open a pull request in Git?
- Do you have access to a cloud account or will you use the course-provided lab project?

Course outcomes

- Plan a configuration comparison for cloud-to-cloud, physical-to-cloud and cloud-to-physical migrations, including what must be proven and what cannot be compared
- Collect running configuration from F5 (iControl REST and tmsh), Citrix NetScaler (NITRO API), Radware Alteon, other load balancers (Anaya LB), Cisco APIC and Terraform state with read-only credentials
- Normalise every platform into one canonical Pydantic model with provenance and explicit unknowns
- Use Claude Code, the Claude Agent SDK and MCP servers so that Claude plans and explains while deterministic Python computes the diff
- Generate severity-classified difference reports, Claude-written explanations and remediation steps grounded in the report
- Gate every consequential action behind GitHub pull request review and a ServiceNow change record, with a full audit trail of prompts, tool calls and decisions

4. Lab environment

Each team gets a lab F5 BIG-IP VE (the 'physical' source), a NetScaler VPX Express instance (the 'cloud' target), recorded fixtures for Radware Alteon, 'Anaya LB' and Cisco APIC, a small Terraform project for a cloud load balancer, a ServiceNow PDI and a GitHub repository with Actions.

Components

GenNoorTech PRIVATE BUILD For more info - https://gennoor.com

Component	Version or edition	Purpose	How to obtain	Notes
F5 BIG-IP Virtual Edition	17.x	Source platform for iControl REST and tmsh collection.	30-day free trial from MyF5 [29].	Preload the sample bigip.conf with virtual servers, pools, monitors, source-addr persistence and one iRule.
NetScaler VPX Express	14.1	Target platform for NITRO collection.	No license file needed; 20 Mbps and 250 SSL sessions [30].	Can run on-premises or in a cloud marketplace image.
Radware Alteon	31.x	CLI export parsing.	Recorded sanitized exports unless the client provides an Alteon VA.	Menu-path syntax per Radware documentation [23].
'Anaya LB'	Unknown	Generic CLI capture adapter.	Client-supplied sanitized output.	No public documentation found under this name; confirm the product with the client.
Cisco APIC	DevNet ACI simulator or recorded responses	Fabric object collection.	Cisco DevNet sandbox or recorded JSON fixtures.	APIC token timeout is 600 seconds by default [22].
Terraform and a cloud load balancer	Terraform 1.x (JSON format_version 1.0)	Cloud-side state and plan checks.	Course lab project for AWS ALB or Azure Application Gateway in a sandbox subscription.	Use terraform show -json [24]; tear down after class to control cost.
ServiceNow Personal Developer Instance	Current Developer Site release	Change records and approval flow.	Free from the ServiceNow Developer Site.	Build the approval flow with a record-based trigger [31].
Claude Code and Claude Agent SDK (Python)	Current releases	Drafting collectors, planning and explanation agents.	Install Claude Code and the claude-agent-sdk package.	Project-scope .mcp.json with ${VAR} credentials [11].
Python toolchain	Python 3.11+, Pydantic 2, pytest, httpx	Collectors, model, diff	Course requirements file.	Use extra='forbid' in canonical models [28].

		engine and tests.		
GitHub with Actions	GitHub.com	Pull requests, branch protection and scheduled runs.	Course organization.	Schedules run in UTC, minimum every 5 minutes [25].

Fixtures and sample data

Fixture	Contents	Used in
Sample bigip.conf and ns.conf pair	Matched configurations with known differences and one iRule.	Configuration comparison as a migration discipline; Normalising into a canonical configuration model
Recorded API and CLI responses	iControl REST, NITRO, Alteon export, APIC JSON.	Collecting configuration from every platform
Bad inventories	Contradictory and incomplete inventories.	Migration planning with Claude
Seeded mismatch set	Hidden persistence timeout and monitor interval mismatches.	The deterministic diff engine; Capstone: end-to-end physical-to-cloud comparison
Terraform lab project	Cloud load balancer definition and a change branch that replaces a listener.	Scheduling, Terraform plan checks and GitHub Actions

Lab safety rules

- Collectors use read-only accounts; no MCP tool exposes write operations.
- Never point the pipeline at production devices or accounts during the course.
- No terraform apply in any course workflow except the instructor's lab setup.
- Credentials only in environment variables, a vault or repository secrets [11].
- Sanitize client fixtures before committing them.

5. Module-by-module content plan

Day 1

Module 1: Configuration comparison as a migration discipline

Goal. Learners define what a configuration comparison must prove for each migration direction and produce the object checklist the pipeline will automate.

Gennoor Tech TRAINING ACADEMY For more info - https://gennoor.com

Outline topics.

- The three directions: cloud-to-cloud, physical-to-cloud, cloud-to-physical, and what each must prove before cut-over
- Why a raw text diff between an F5 and a NetScaler configuration is worthless; semantic objects versus syntax
- Scoping the comparison: virtual servers, pools, monitors, persistence, SSL profiles, policies, routing and NAT
- The course pattern: Claude plans, drafts and explains; deterministic code executes; a person approves; everything is logged

Learning objectives.

- Learners can state what must be proven before cut-over for cloud-to-cloud, physical-to-cloud and cloud-to-physical moves.
- Learners can explain why a text diff between vendor dialects is meaningless and what a semantic comparison compares instead.
- Learners can scope a comparison by object type and list what cannot be compared.
- Learners can describe the course pattern: Claude plans and explains, code computes, a person approves, everything is logged.

Three directions, three proofs

- Physical to cloud: every VIP, pool, monitor, persistence and certificate on the appliance has a mapped target object or a documented gap.
- Cloud to cloud: AWS ALB listeners, rules and target groups [4] versus Azure Application Gateway listeners, routing rules, backend settings and pools [5]; constructs differ even where intent is the same.
- Cloud to physical: cloud defaults (for example ALB round robin [4], App Gateway HTTP/1.1 to backends [5]) must be made explicit on the appliance.
- The deliverable for each direction is the same: a severity-classified difference report plus a list of things that cannot be compared.

Why text diff fails

- BIG-IP describes a virtual server with destination, pool, profiles, persist, rules, policies and source-address-translation [1]; NetScaler uses lbvserver, servicegroup, monitor and policy bindings in ns.conf [2][16].
- Same-vendor diff tools exist (NetScaler diff ns config compares two ns.conf files [2]; Alteon diff flash shows pending versus saved [23]) but they only compare one dialect.
- Defaults hide meaning: F5 source-address persistence defaults to 180 seconds [18] while NetScaler's lbvserver timeout defaults to 2 [16]; an omitted line on one side is not 'no difference'.
- Units differ by platform; normalize before comparing.

Scoping the comparison

- In scope: virtual servers, pools and members, monitors, persistence, SSL profiles and certificates, policies (iRules, LTM policies, responder or rewrite), routing and NAT.
- Mark each object type comparable, comparable with mapping, or not comparable (for example iRule logic versus a responder policy is reviewed by a person).
- Capture dependencies: monitors before pools, pools before virtual servers.
- Record which runtime state is excluded (statistics, connection counts) because it is not configuration.

The course pattern

- Claude plans, drafts and explains; deterministic Python collects, normalizes and diffs; a person approves; every prompt and tool call is logged.
- MCP calls for a human in the loop who can deny tool invocations [14]; the pipeline exposes no write tools to Claude at all.
- The module 1 checklist is the acceptance standard for the capstone.
- Each later module ends with a deliverable that slots into this pattern: collectors, model, plan, diff, explanation, approval, pipeline.

Instructor demo.

1. Show a short bigip.conf virtual server stanza and the equivalent ns.conf lines side by side [1][2].
2. Run a plain text diff on them and show that every line differs.
3. Run diff ns config on two NetScaler files to show what a same-vendor diff can and cannot do [2].
4. Point out the persistence default trap (180 seconds on F5 [18], default 2 on NetScaler [16]).
5. Build the first rows of the object checklist with the class.
6. Show the target capstone approval package so learners see the end state.

Hands-on lab. Take a sample physical F5 configuration and a hand-built cloud NetScaler configuration and write, by hand, the comparison plan and object checklist that the pipeline must later automate.

1. Read the sample bigip.conf and list every virtual server, pool, monitor, persistence profile and iRule.
2. Read the sample ns.conf and list every lbvserver, servicegroup, monitor and policy binding.
3. Match objects by intent (VIP address and port) rather than by name.
4. For each pair, list the fields that must match and the unit conversions needed.
5. Mark each object type comparable, comparable with mapping, or not comparable.
6. Write the 'cannot be compared' list with a reason for each entry.
7. Write the comparison plan: direction, scope, evidence needed, approvers.
8. Review a partner's checklist and add anything they missed.

Lab success criteria.

- Every virtual server on the F5 has a matched target or is listed as unmatched.

- Every compared field has a defined unit and default.
- The 'cannot be compared' list exists and is justified.
- The plan names who approves the result.

Deliverable. Comparison plan template and object checklist.

Common pitfalls.

- Matching objects by name; names rarely survive a migration.
- Ignoring defaults that are not printed in the configuration.
- Treating runtime statistics as configuration differences.
- Leaving iRule-to-policy translation to the diff engine instead of human review.

Knowledge check.

1. Why is a raw text diff between an F5 and a NetScaler configuration useless? Answer: The dialects describe the same intent with different objects, names, defaults and units [1][2].
2. What is F5's default source-address persistence timeout? Answer: 180 seconds [18].
3. What does NetScaler's diff ns config compare? Answer: Two NetScaler configuration files [2].
4. What are the three classes in the object checklist? Answer: Comparable, comparable with mapping, not comparable.

Slides and assets to prepare.

- Sample bigip.conf and ns.conf pair.
- Object checklist template.
- Comparison plan template per direction.
- Slide: three migration directions and what each must prove.

Sources for this module. 1, 2, 4, 5, 14, 16, 18, 23

Module 2: Setting up Claude Code, the Agent SDK and MCP for the workflow

Goal. Learners stand up a workbench where Claude can call read-only collector tools through MCP without ever seeing a credential.

Outline topics.

- Installing Claude Code and configuring a project with CLAUDE.md conventions for the comparison repository
- Claude Agent SDK basics: sessions, tool definitions, structured JSON outputs and when to use the SDK instead of the interactive CLI
- Building a minimal MCP server that exposes read-only collector tools (get_virtual_servers, get_pools) to Claude
- Secrets handling: credentials in environment variables and vaults, never in prompts or repositories

Learning objectives.

- Learners can install Claude Code and write a CLAUDE.md for the comparison repository.
- Learners can explain when to use the Agent SDK instead of the interactive CLI.
- Learners can build and register an MCP server exposing get_virtual_servers and get_pools.
- Learners can keep secrets in environment variables or a vault referenced from .mcp.json.

Claude Code in the repository

- Create ./CLAUDE.md with /init, then add conventions: canonical model location, never edit fixtures, diff engine must not call a model [10].
- CLAUDE.md is context, not enforcement; use permission rules or hooks for anything that must be blocked [10].
- Keep it short and import longer standards with @path [10].
- Commit project settings and .mcp.json; keep personal overrides local.

Agent SDK versus CLI

- CLI for interactive work: writing collectors, reviewing diffs.
- Agent SDK for the pipeline: planning and explanation agents run unattended in CI.
- Define in-process tools with @tool and create_sdk_mcp_server and list them in allowed_tools as mcp__{server}__{tool} [12].
- Remove built-ins such as Bash and Write from the explanation agent with the tools option [12].
- Use structured output for any result the pipeline consumes [13].

A read-only collector MCP server

- Expose get_virtual_servers(device) and get_pools(device), implemented with GET calls only.
- Register with claude mcp add --transport stdio collectors -- python servers/collectors.py at project scope so .mcp.json is shared [11].
- Clients discover tools through tools/list and call them through tools/call; return errors with isError [14].
- Return outputSchema-conformant structured results so callers can validate them [14].

Secrets

- Reference credentials as ${F5_USER} and ${F5_PASS} in .mcp.json; the values come from the environment or a vault agent [11].
- Tools never echo credentials or auth tokens in results.
- Use read-only roles on every device and API.
- Check the transcript for secrets as part of the lab sign-off.

Instructor demo.

1. Run /init and edit CLAUDE.md live [10].
2. Write the collectors MCP server with two GET-backed tools.

3. Register it at project scope and show .mcp.json with ${VAR} placeholders [11].
4. Ask Claude to list virtual servers on the lab F5 and show the tools/call in the transcript [14].
5. Show the same call from an Agent SDK script with allowed_tools limited to the two tools [12].
6. Search the transcript for the password and show it is absent.

Hands-on lab. Stand up the workbench, register the read-only MCP server, and have Claude list the virtual servers on the lab F5 through tool use without ever seeing a password.

1. Install Claude Code and clone the course repository.
2. Generate and edit CLAUDE.md with the repository conventions [10].
3. Implement get_virtual_servers using GET /mgmt/tm/ltm/virtual [15].
4. Implement get_pools using GET /mgmt/tm/ltm/pool.
5. Register the server with claude mcp add at project scope [11].
6. Export credentials in your shell only; confirm claude mcp list shows no warnings.
7. Ask Claude to list the virtual servers and their destinations.
8. Run the Agent SDK version of the same query and save both transcripts.

Lab success criteria.

- Claude lists the lab F5 virtual servers through the MCP tool.
- No credential appears in the repository or either transcript.
- The server exposes only GET-backed tools.
- CLAUDE.md states the 'diff engine never calls a model' rule.

Deliverable. Configured workbench with a working read-only MCP collector.

Common pitfalls.

- Hard-coding credentials in .mcp.json instead of ${VAR} [11].
- Exposing a generic 'run any REST call' tool.
- Expecting CLAUDE.md to block actions [10].
- Using an admin role for the collector because it was quicker.

Knowledge check.

1. How do you share an MCP server definition with the team? Answer: Add it with --scope project so it is written to .mcp.json in the repository [11].
2. When should the pipeline use the Agent SDK rather than the CLI? Answer: For unattended, scripted runs such as CI planning and explanation jobs.
3. How are SDK tools named in allowed_tools? Answer: mcp__{server}__{tool} [12].
4. Which F5 endpoint lists virtual servers? Answer: GET /mgmt/tm/ltm/virtual [15].

Slides and assets to prepare.

- CLAUDE.md template.
- collectors.py MCP server starter.

Gennoor Tech
TRAIN. INNOVATE. BUILD.
For more info https://gennoor.com

- .mcp.json example with ${VAR} placeholders.
- Architecture slide: Claude, SDK, MCP, collectors, devices.

Sources for this module. 10, 11, 12, 13, 14, 15

Module 3: Collecting configuration from every platform

Goal. Learners build idempotent, read-only collectors for F5, NetScaler, Radware Alteon, other load balancers, Cisco APIC and Terraform state that keep the raw payload as evidence.

Outline topics.

- F5: iControl REST (/mgmt/tm/ltm) and tmsh list output; Citrix NetScaler: NITRO API config resources
- Radware Alteon via Cyber Controller and CLI exports; other load balancers (Anaya LB) through CLI capture adapters
- Cisco APIC REST API for fabric objects and Terraform state (terraform show -json) for cloud-side resources
- Collector contract: idempotent, read-only, timestamped, raw payload retained as evidence

Learning objectives.

- Learners can collect F5 configuration with iControl REST and tmsh list output.
- Learners can collect NetScaler configuration with NITRO config resources.
- Learners can capture Alteon and other load balancer configuration through CLI exports.
- Learners can collect APIC objects and Terraform state as JSON.
- Learners can enforce the collector contract.

F5 BIG-IP

- iControl REST: /mgmt/tm/ltm/virtual and /mgmt/tm/ltm/virtual/~Common~<name>, keyed by name, partition and subPath [15].
- Collect pools, members, monitors, profiles (including persistence) and iRules as separate collections and keep the raw JSON.
- tmsh list ltm virtual output is the fallback when REST is unavailable; properties match the ltm virtual man page [1].
- Record partitions; objects outside /Common are often missed.

NetScaler

- GET /nitro/v1/config/lbvserver with pagesize and pageno for large estates; ?count=yes to check completeness [16].
- Collect servicegroup, monitor, binding and policy resources and keep each response.
- Authenticate with the NITRO_AUTH_TOKEN cookie or X-NITRO-USER/X-NITRO-PASS, never both [16].

- Capture show ns runningConfig as a text artifact and compare it with show ns ns.conf to detect unsaved changes [2].

Radware Alteon and other load balancers

- Alteon configuration uses menu paths such as /cfg/slb/virt <vsid>/service <vport>; diff flash shows unsaved differences [23].
- Collect a full configuration dump through the CLI and parse it with a deterministic parser per menu path.
- For 'Anaya LB', build a CLI capture adapter (fixed command list, prompt detection, timeout) against recorded output.
- Without lab appliances, use client-supplied sanitized exports as fixtures.

Cisco APIC and Terraform

- APIC: POST aaaLogin, then GET /api/mo/uni/tn-<tenant>.json?query-target=subtree for tenant objects; page with page-size [22].
- Terraform: terraform show -json <state or plan file> returns format_version '1.0' and values or resource_changes [24].
- Cloud load balancer resources from state (ALB listeners and target groups [4], App Gateway components [5]) become the cloud side of the comparison.
- Never run terraform apply from a collector.

Collector contract

- Read-only: GET and show commands only (APIC login excepted).
- Idempotent: two runs produce the same configuration objects.
- Timestamped and attributed: device, endpoint, collected_at, collector version.
- Raw payload saved under runs/<timestamp>/raw/ and referenced from every normalized field.

Instructor demo.

1. Collect virtual servers from the lab F5 through iControl REST and save the raw JSON [15].
2. Collect lbvservers from the NetScaler VPX through NITRO with paging and count [16].
3. Show show ns runningConfig versus show ns ns.conf for unsaved changes [2].
4. Parse a recorded Alteon /cfg/slb/virt export [23].
5. Query an APIC tenant subtree in the DevNet simulator [22].
6. Run terraform show -json on the lab state and point out format_version [24].

Hands-on lab. Write Python collectors for F5 iControl REST and NetScaler NITRO against the lab devices, then Radware Alteon CLI export and Cisco APIC REST collectors against recorded API fixtures, with Claude drafting the code and you reviewing every diff.

1. Ask Claude Code to draft the F5 collector; review and correct the diff before accepting.
2. Run it against the lab F5 and save raw JSON per collection [15].

3. Draft and review the NetScaler collector with paging and count checks [16].
4. Draft and review the Alteon parser against the recorded export [23].
5. Draft and review the APIC collector against recorded responses [22].
6. Add a Terraform state reader using terraform show -json [24].
7. Run every collector twice and confirm idempotency.
8. Write unit tests that load each fixture.

Lab success criteria.

- Four platform families collected with raw evidence saved.
- Each collector is read-only and documented.
- Two runs produce identical configuration objects.
- You reviewed and can explain every line Claude drafted.

Deliverable. Collector library with raw evidence capture for four platform families (F5, NetScaler, Radware Alteon, Cisco APIC).

Common pitfalls.

- Missing objects in non-Common partitions on F5.
- Not paging NITRO results and silently truncating large estates [16].
- Comparing running configuration that was never saved [2].
- Letting a collector call terraform apply or plan with write credentials.

Knowledge check.

1. How do you check that a NITRO collection is complete? Answer: Compare the collected number with GET ...?count=yes [16].
2. Which command renders Terraform state as JSON? Answer: terraform show -json <file> [24].
3. Why save raw payloads? Answer: Every normalized field and every diff line must trace back to evidence.
4. Which Alteon command shows unsaved configuration differences? Answer: diff flash [23].

Slides and assets to prepare.

- Collector contract one-pager.
- Recorded Alteon and APIC fixtures.
- Lab Terraform project for the cloud load balancer.
- Endpoint cheat sheet per platform.

Sources for this module. 1, 2, 4, 5, 15, 16, 22, 23, 24

Module 4: Normalising into a canonical configuration model

Goal. Learners design one canonical Pydantic model with provenance on every field and explicit unknowns, and normalize F5 and NetScaler configurations into it.

Outline topics.

- Designing the canonical Pydantic model: VirtualServer, Pool, Member, Monitor, Persistence, SSLProfile, Policy
- Mapping tables per platform and the rule that unmapped fields are marked unknown, never dropped silently
- Provenance on every field: source platform, object name, collection time, raw reference
- Validation tests for round-tripping fixtures through the model

Learning objectives.

- Learners can design canonical classes for VirtualServer, Pool, Member, Monitor, Persistence, SSLProfile and Policy.
- Learners can write per-platform mapping tables with unit conversions and defaults.
- Learners can attach provenance to every field.
- Learners can test round-tripping fixtures through the model.

Designing the canonical model

- VirtualServer: address, port, protocol, pool ref, persistence ref, ssl refs, policies, vlans or networks, snat mode.
- Pool: members, lb method, monitors, priority groups and minimum active members (F5 priority-group 0 to 65535, min-active-members default 0) [19].
- Persistence: type and timeout in seconds; F5 source-addr defaults to 180 s [18]; NetScaler persistencetype and timeout (default 2) [16][17].
- Use Pydantic with extra='forbid' so unexpected keys fail loudly instead of being ignored (the default is 'ignore') [28].
- Validate with model_validate() and serialize with model_dump_json() [28].

Mapping tables and unknowns

- One table per platform: source field, canonical field, conversion, default if absent.
- LB method defaults differ: F5 pools default to round-robin [19]; NetScaler lbvserver defaults to LEASTCONNECTION [16]; ALB defaults to round robin [4].
- Unmapped fields go into an unknowns list with the raw value, never dropped.
- Profiles with no direct equivalent (for example F5's Fast L4 profile [20]) are recorded as unknowns for the planner.

Provenance

- Every field carries source platform, object name, collection time and raw reference (file and JSON pointer).
- Defaults applied by the normalizer are marked as 'default', not as observed.
- Provenance is what the explanation agent cites later.
- Keep the canonical model versioned (for example v1.3) and record the version in every report.

Validation tests

- Round-trip each fixture: raw to canonical to JSON and back to canonical; objects must be equal.
- Test that unknown fields land in unknowns.
- Test unit conversion for persistence timeouts and monitor intervals.
- Run the tests in CI from day 1.

Instructor demo.

1. Show the canonical model classes and the extra='forbid' setting [28].
2. Normalize the lab F5 virtual server with source-addr persistence and show the 180-second default applied with provenance 'default' [18].
3. Normalize the NetScaler lbvserver and show persistencetype and timeout mapping [16].
4. Show a Fast L4 profile landing in unknowns [20].
5. Ask Claude to explain each unknown and show it citing provenance.
6. Run the round-trip tests.

Hands-on lab. Build the normaliser for F5 and NetScaler, run both lab configurations through it, and inspect the unknowns list with Claude explaining each one.

1. Write the canonical Pydantic classes with provenance fields [28].
2. Write the F5 mapping table including persistence and pool defaults [18][19].
3. Write the NetScaler mapping table including lbmethod and timeout defaults [16][17].
4. Implement the normalizers and the unknowns collector.
5. Normalize both lab configurations.
6. Ask Claude to explain each unknown, citing provenance only.
7. Write round-trip and conversion tests.
8. Commit the model with a version number.

Lab success criteria.

- Both configurations normalize without validation errors.
- Every field has provenance.
- Every unmapped field appears in unknowns.
- Round-trip tests pass.

Deliverable. Canonical model, mapping tables and normalisation test suite.

Common pitfalls.

- Using Pydantic's default extra='ignore' and silently losing fields [28].
- Comparing NetScaler timeout values to F5 seconds without confirming units on the appliance.
- Recording a normalizer default as if it had been observed on the device.
- Changing the canonical model without bumping its version.

Knowledge check.

1. What does extra='forbid' do in Pydantic? Answer: Rejects data with keys that do not map to fields [28].
2. What is F5's default pool load balancing method? Answer: Round-robin [19].
3. What is NetScaler's default lbmethod for an lbvserver? Answer: LEASTCONNECTION [16].
4. What four facts make up provenance? Answer: Source platform, object name, collection time, raw reference.

Slides and assets to prepare.

- Canonical model class diagram.
- Mapping table templates for F5 and NetScaler.
- Unknowns report template.
- Round-trip test starter.

Sources for this module. 4, 16, 17, 18, 19, 20, 28

Day 2

Module 5: Migration planning with Claude

Goal. Learners build a planning agent that turns a source inventory and target capabilities into a structured migration plan that cites the inventory and states what cannot be mapped.

Outline topics.

- Prompting Claude with the source inventory and target platform capabilities to draft a migration plan as structured JSON
- Required fields: object mapping, feature gap, dependency order, risk, confidence, owner, and citation to the inventory
- Handling incomplete or contradictory inventories: the plan must say what cannot be mapped
- Reviewing and versioning plans in Git as the first human checkpoint

Learning objectives.

- Learners can prompt Claude with inventory and target capabilities to produce a JSON plan.
- Learners can require mapping, gap, dependency order, risk, confidence, owner and citation fields.
- Learners can test the planner with incomplete and contradictory inventories.
- Learners can review and version plans in Git.

Inputs to the planner

- The canonical inventory and unknowns from module 4.
- A target capability sheet: for NetScaler, persistence types and backup virtual servers [16][17]; responder policies for redirects [21]; for cloud targets, ALB and App Gateway components [4][5].

Gennoor Tech
INNOVATE. BUILD.
For more info: https://gennoor.com

- Vendor migration tools as reference points: F5 ACC to AS3 [6], F5 Journeys for UCS-based migrations and parity gaps [7], NetScaler Configuration Builder and its known blind spots [3].
- Claude reads these through tools, not pasted whole configurations.

The plan schema

- Each step: source object id, target object type, mapping, feature gap, dependency order, risk, confidence, owner, citations to inventory ids.
- Return the plan as structured output validated against the schema [13].
- Dependency order follows object dependencies: monitors, then pools or service groups, then virtual servers.
- Gaps example: an iRule redirect maps to a responder action and policy bound to the lbvserver, reviewed by a person [21].

Handling bad inventories

- Missing values (for example no persistence timeout recorded) must appear as unknown, not as an assumed default.
- Contradictions (two virtual servers claiming the same address and port) must be flagged and left unmapped.
- Priority groups on F5 pools [19] need a design decision on the target, such as a backup virtual server on NetScaler [17]; the planner proposes, a person decides.
- Named expressions and responder HTML pages are not detected by NetScaler's Configuration Builder [3]; the plan must list them as manual items.

Review and versioning

- Plans are committed to Git as JSON plus a rendered summary.
- A pull request is the first human checkpoint; reviewers comment on steps, not formatting.
- Plan versions are referenced by the later diff and approval package.
- Reject plans whose citations do not resolve to inventory ids before the review starts.

Instructor demo.

1. Show the inventory, unknowns and capability sheet the planner receives.
2. Run the planner and show the structured plan with citations [13].
3. Show an iRule-to-responder mapping marked for human review [21].
4. Feed a contradictory inventory and show the conflict flagged.
5. Open a pull request with the plan.

Hands-on lab. Generate a physical-to-cloud plan for the lab inventory, then feed Claude a deliberately contradictory inventory and verify it flags the conflict instead of inventing a mapping.

1. Write the PlanStep and Plan Pydantic models.
2. Write the capability sheet for the cloud NetScaler target [16][17][21].
3. Write the planner prompt with citation and unknowns rules.

4. Run the planner with output_format [13].
5. Validate every citation against the inventory.
6. Run the contradictory inventory and check for a flagged conflict.
7. Run an inventory with a missing persistence timeout and check it is unknown.
8. Commit the plan and open a pull request.

Lab success criteria.

- The plan validates and every step cites inventory ids.
- Gaps and manual items are listed explicitly.
- The contradictory inventory produces a flagged conflict, not a mapping.
- The plan is under review in a pull request.

Deliverable. Migration planning agent with JSON schema and review checklist.

Common pitfalls.

- Pasting whole configurations into the prompt instead of tool access.
- Accepting high confidence on steps that map logic (iRules) automatically.
- Forgetting manual items the vendor tool cannot detect [3].
- Plans not versioned and not tied to a collection run.

Knowledge check.

1. What must the planner do with a value missing from the inventory? Answer: Mark it unknown, never assume a default.
2. What does F5 ACC convert configuration into? Answer: AS3 or Declarative Onboarding declarations [6].
3. Which NetScaler items does Configuration Builder not detect? Answer: Named expressions and responder HTML pages [3].
4. Which NetScaler feature replaces a simple redirect iRule? Answer: A responder action and policy bound to the virtual server [21].

Slides and assets to prepare.

- Plan schema and review checklist.
- Target capability sheets (NetScaler, ALB, App Gateway).
- Contradictory and incomplete inventory fixtures.
- Slide: vendor migration tools and their limits.

Sources for this module. 3, 4, 5, 6, 7, 13, 16, 17, 19, 21

Module 6: The deterministic diff engine

Goal. Learners implement a Python diff engine that compares canonical models field by field, classifies differences by severity and produces a JSON report without ever calling a model.

Gennoor Tech TRAINING INNOVATE BUILD For more info - https://gennoor.com

Outline topics.

- Field-by-field comparison of canonical models in Python; ordering, set semantics and tolerated differences
- Severity classification rules: blocking, warning, informational, and platform-specific exemptions
- Machine-readable diff report (JSON) plus a human summary; the engine never calls a model
- Unit tests with pytest, including seeded mismatches that must always be caught

Learning objectives.

- Learners can implement field-by-field comparison with ordering and set semantics.
- Learners can define severity rules and exemptions.
- Learners can produce a machine-readable report and a human summary.
- Learners can prove with tests that seeded mismatches are always caught.

Comparison semantics

- Match objects by intent key (address, port, protocol) produced in module 4.
- Compare lists as sets where order does not matter (pool members) and as sequences where it does (policy rule order).
- Compare normalized values only, after unit conversion, with defaults marked.
- Tolerances are explicit, versioned rules, not ad hoc code.

Severity classification

- Blocking: changes traffic behavior at cut-over (persistence type or timeout, pool membership, VIP port, certificate).
- Warning: likely behavior change under load or failure (monitor interval, priority groups versus backup design) [19].
- Informational: cosmetic or platform-inherent (object names, descriptions).
- Exemptions are listed per platform pair with a reason, for example backend HTTP/1.1 on App Gateway [5].

The report

- JSON: run id, model version, source and target, per-object diffs with field, source value, target value, severity, provenance refs.
- Human summary: counts per severity and the blocking list.
- The engine never calls a model; Claude reads the report later through tools.
- Report schema is versioned and validated with Pydantic [28].

Testing

- pytest suites with fixtures for matched, mismatched and missing objects.
- Seeded mismatch: source-addr persistence 300 s on the source versus 180 s on the target must be blocking.
- Property-style tests: shuffling member order never creates a difference.

GenNoorTech
For more info: https://gennoor.com

- Tests run on every pull request.

Instructor demo.

1. Walk through the engine's comparison of two VirtualServer objects.
2. Show the severity rules file.
3. Seed a persistence timeout mismatch into the target and run the engine.
4. Show the JSON report line and the summary.
5. Run the pytest suite and show the seeded test.

Hands-on lab. Implement the diff engine, seed a hidden persistence timeout mismatch into the target configuration, and prove the test suite catches it.

1. Implement match_objects() by intent key.
2. Implement compare_fields() with set and sequence semantics.
3. Write the severity rules file and loader.
4. Implement report generation validated by a Pydantic schema [28].
5. Write fixtures and tests for matched, mismatched and missing cases.
6. Seed the persistence timeout mismatch in a hidden fixture.
7. Run the suite and confirm the seeded test fails before the rule exists and passes after.
8. Commit through a pull request.

Lab success criteria.

- The engine produces a valid JSON report.
- The seeded mismatch is reported as blocking.
- Shuffled member order produces no difference.
- No model call exists anywhere in the engine code.

Deliverable. Tested diff engine producing severity-classified JSON reports.

Common pitfalls.

- Comparing unnormalized values.
- Severity decided in code branches instead of a rules file.
- Treating platform defaults as differences when both sides are equivalent after normalization.
- Letting the engine call Claude 'just to summarize'.

Knowledge check.

1. Why does the diff engine never call a model? Answer: So the diff is deterministic, repeatable and testable; Claude only explains it.
2. Give one blocking and one informational difference. Answer: Blocking: persistence timeout or pool membership. Informational: object name.
3. How do you prove the engine catches a mismatch? Answer: A seeded mismatch fixture with a test that must fail if it is missed.

Genmoor Tech INNOVATE. BUILD. For more info - https://genmoor.com

4. Why compare pool members as a set? Answer: Member order does not change behavior, so order differences are not real differences.

Slides and assets to prepare.

- Diff engine starter.
- Severity rules file template.
- Seeded mismatch fixtures.
- Report JSON schema.

Sources for this module. 5, 19, 28

Module 7: Claude explains the differences

Goal. Learners build an explanation agent that reads the diff report through tools and writes explanations, drift summaries and remediation steps with every claim tied to a report line.

Outline topics.

- Giving Claude the diff report through tool use (get_diff, get_object) rather than pasting whole configurations
- Structured explanation output: what differs, why it matters for traffic, recommended remediation, confidence, citation to the diff line
- Drift summaries for operations managers and remediation steps for engineers from the same report
- Guarding against hallucinated objects: every referenced object must exist in the report

Learning objectives.

- Learners can expose get_diff and get_object tools to Claude.
- Learners can require structured explanations with citations to report lines.
- Learners can produce two audiences' outputs from one report.
- Learners can reject any output that references objects not in the report.

Tool access, not pasting

- Define get_diff(run_id, severity) and get_object(run_id, object_id) tools with input schemas [8] [12].
- Claude responds with tool_use, your code returns tool_result [8].
- Use strict: true on custom tool definitions when calling the API directly so calls match the schema [8].
- Remove file and shell built-ins from the explanation agent [12].

Structured explanation output

- Per difference: what differs, why it matters for traffic, recommended remediation, confidence, citation to the report line.

GenNoor Tech TRAINING INNOVATE BUILD
For more info- https://gennoor.com

- Example: source-address persistence shorter on the target means sessions may move to another server after the shorter idle time; remediation sets the target timeout to match.
- Use output_format with a Pydantic schema [13].
- Unknown impacts are stated as unknown.

Two audiences

- Operations managers: drift summary with counts, blocking items and cut-over risk.
- Engineers: remediation steps with target CLI or API objects named, never executed.
- Both are generated from the same report and cite the same lines.
- Use the same schema with an audience field so tests can check that both outputs cover the same blocking items [13].

Guarding against hallucinated objects

- A deterministic check confirms every object id and report line referenced exists.
- Outputs that fail the check are rejected and regenerated once, then escalated.
- Tool annotations are not trusted for safety; tools themselves are read-only [14].
- Log rejected outputs with the reason so prompt changes can be evaluated against them.

Instructor demo.

1. Show the get_diff and get_object tool definitions [12].
2. Run the explanation agent on the seeded mismatch and show the tool calls [8].
3. Show the structured explanation with citation diff#L3 style references [13].
4. Inject a fake object id into the output and show the check rejecting it.
5. Generate the manager summary and engineer remediation from the same report.

Hands-on lab. Build the explanation agent with the Agent SDK, run it on the day's diff report, and verify every statement traces to a report line.

1. Implement get_diff and get_object as SDK tools [12].
2. Write the Explanation Pydantic schema with citations [13].
3. Write the prompt for engineers and the prompt for managers.
4. Run on the module 6 report.
5. Implement the citation and object existence check.
6. Test the check with a deliberately bad output.
7. Render the remediation report template.
8. Commit outputs with the run id.

Lab success criteria.

- Every explanation cites a report line that exists.
- No output references an object outside the report.
- Manager and engineer outputs agree on blocking items.

- Remediation steps are proposals only; nothing is executed.

Deliverable. Explanation agent and remediation report template.

Common pitfalls.

- Pasting the full report into the prompt and losing traceability.
- Allowing the agent to propose remediation it then executes.
- Accepting plausible explanations without the existence check.
- Different counts in manager and engineer outputs.

Knowledge check.

1. What stop_reason indicates Claude wants to call your tool? Answer: tool_use [8].
2. What does strict: true do on a tool definition? Answer: Ensures tool calls always match the schema [8].
3. How do you catch a hallucinated object? Answer: A deterministic check that every referenced id exists in the report.
4. Why give Claude tools instead of the whole report? Answer: Smaller context, targeted reads and citations that map to specific lines.

Slides and assets to prepare.

- Explanation schema.
- Remediation report template.
- Existence check script.
- Slide: tool round trip.

Sources for this module. 8, 12, 13, 14

Day 3

Module 8: Guardrails, approvals and audit for configuration comparison

Goal. Learners wire the pipeline so that every comparison becomes a reviewed pull request and a ServiceNow change routed for approval, with no write capability exposed to Claude and a full audit log.

Outline topics.

- Read-only credentials for all collectors; no write capability exposed to Claude through MCP
- Opening a ServiceNow change record with the Table API and attaching the report; Flow Designer approval routing to a named approver
- Publishing each comparison as a GitHub pull request with the report, evidence and plan for review
- Audit log of prompts, tool calls, model outputs and human decisions, retained alongside the raw evidence

GennoorTech TRAIN · INNOVATE · BUILD For more info - https://gennoor.com

Learning objectives.

- Learners can verify that all collectors use read-only credentials and no MCP tool can write.
- Learners can create a ServiceNow change through the Table API and attach the report.
- Learners can publish a comparison as a GitHub pull request with protection rules.
- Learners can keep an audit log of prompts, tool calls, outputs and human decisions.

No write path for Claude

- List every MCP tool and confirm each is GET or show only [14].
- Deny rules and the tools option keep shell and file writes away from pipeline agents [12].
- Remediation is a document, applied by a person through normal change control.
- MCP guidance: human in the loop and confirmation for sensitive operations [14].

ServiceNow change record

- Create a change with POST /api/now/table/change_request; POST inserts one record at a time [27].
- Store the run id, report summary and pull request link in the change description; attach the report.
- A record-based Flow Designer trigger on the new change routes it for approval to the named approver; expect queue delay [31].
- Read state back with GET on the change before allowing any remediation step [27].

GitHub pull request

- Each comparison run opens a pull request containing the plan, report, explanation and evidence index.
- Protect main: required approving reviews (optionally from code owners), required status checks, stale approvals dismissed on new commits [26].
- Reviewers approve the comparison, not the remediation.
- The pull request and change cross-link.

Audit log

- Log prompts, tool calls with inputs, tool outputs (hashed), model outputs and human decisions with timestamps.
- MCP clients should log tool usage for audit [14].
- Store the audit log with the raw evidence, outside the agent's write scope.
- A blocked remediation attempt is logged with who, what and why.

Instructor demo.

1. List MCP tools and show none can write [14].
2. Create a change through the Table API and show it in the PDI [27].
3. Show the Flow Designer approval routing firing [31].

Gemnoor Tech TRAINING ACADEMY For more info - https://gemnoor.com

4. Open the pull request and show branch protection blocking merge without approval [26].
5. Attempt an unapproved remediation and show the block and audit entry.

Hands-on lab. Wire the pipeline to create a ServiceNow change via the Table API and a GitHub pull request, then attempt an unapproved remediation and confirm it is blocked and logged.

1. Audit your MCP tool list for write capability.
2. Implement create_change() with POST on change_request [27].
3. Build the Flow Designer flow that routes the change to the approver [31].
4. Implement open_pr() with the plan, report, explanation and evidence index.
5. Enable branch protection with required reviews and checks [26].
6. Implement the remediation gate that checks change approval state through GET [27].
7. Attempt remediation before approval and capture the block.
8. Review the audit log for the full run.

Lab success criteria.

- A change record exists, linked to the pull request.
- The pull request cannot merge without review and green checks.
- The unapproved remediation is blocked and logged.
- The audit log covers every prompt, tool call and decision.

Deliverable. Approval and audit workflow with blocked-action evidence.

Common pitfalls.

- Giving the pipeline service account write roles on devices 'for later'.
- Approving in GitHub and assuming the change is approved too.
- Not handling Flow Designer delay before checking approval [31].
- Audit logs that the agent can modify.

Knowledge check.

1. Which Table API call creates a change request? Answer: POST /api/now/table/change_request [27].
2. Which branch protection setting forces re-approval after new commits? Answer: Dismissing stale approvals when new commits are pushed [26].
3. What does the pipeline do before any remediation step? Answer: Reads the change record and requires an approved state.
4. Why is remediation a document rather than an action? Answer: Claude has no write path; people apply changes through change control.

Slides and assets to prepare.

- Change record template.
- Flow Designer flow export.

Gennoor Tech
INNOVATE · BUILD
For more info https://gennoor.com

- Branch protection checklist.
- Audit log schema.

Sources for this module. 12, 14, 26, 27, 31

Module 9: Scheduling, Terraform plan checks and GitHub Actions

Goal. Learners run comparisons on a schedule and on Terraform pull requests, comparing the Terraform plan with live collected state and publishing results to the pull request and ServiceNow.

Outline topics.

- Running the comparison on a schedule in GitHub Actions and on pull requests that change Terraform code
- Comparing terraform plan output against live collected state before apply; flagging drift on both sides
- Notifications and report publishing: PR comments, ServiceNow updates, archived reports
- Operating the pipeline: failures, retries, fixture refresh and runbook

Learning objectives.

- Learners can schedule comparisons with GitHub Actions cron.
- Learners can run terraform plan in CI and read its JSON output.
- Learners can compare planned cloud load balancer changes with live collected state.
- Learners can publish results and operate the pipeline with a runbook.

Scheduling

- on: schedule with POSIX cron runs in UTC by default on the latest commit of the default branch; minimum interval 5 minutes [25].
- Combine schedules and read github.event.schedule to vary behavior [25].
- Also trigger on pull_request for paths that change Terraform code.
- Scheduled runs are read-only and post results; they never remediate.

Terraform plan in CI

- terraform plan -out=tfplan -detailed-exitcode: exit 0 no changes, 1 error, 2 changes present [9].
- terraform show -json tfplan gives resource_changes with change.actions and before and after values [24].
- resource_drift lists changes Terraform detected outside its control [24].
- A -refresh-only plan shows state drift without proposing configuration changes [9].

Plan versus live state

- Normalize the planned after-values for the cloud load balancer into the canonical model.
- Compare with the live collected state; differences on both sides are drift.

- Flag planned replacements (['delete','create']) on listeners or target groups as blocking for review [24].
- Feed results to the explanation agent like any other report.

Publishing and operations

- Comment the summary on the pull request and update the ServiceNow change [27].
- Archive reports and evidence per run.
- Runbook: collector failures, token expiry, retries with backoff, fixture refresh.
- Alert on scheduled run failures, not only on differences.

Instructor demo.

1. Show the workflow file with schedule and pull_request triggers [25].
2. Run terraform plan with -detailed-exitcode and show exit code 2 [9].
3. Show terraform show -json output and a resource_changes entry [24].
4. Compare the plan with live ALB or App Gateway state and show a drift line.
5. Show the pull request comment and the ServiceNow update [27].

Hands-on lab. Build the GitHub Actions workflow that runs the comparison, compares a Terraform plan against the live cloud load balancer, and comments the result on the pull request.

1. Write the workflow with schedule and pull_request triggers [25].
2. Add terraform init and plan with -out and -detailed-exitcode [9].
3. Convert the plan with terraform show -json [24].
4. Normalize planned values and run the diff against live state.
5. Run the explanation agent on the result.
6. Comment the summary on the pull request.
7. Update the ServiceNow change with the result [27].
8. Write the runbook.

Lab success criteria.

- The workflow runs on schedule and on Terraform pull requests.
- Planned replacements are flagged blocking.
- Drift between plan and live state is reported with provenance.
- Results appear on the pull request and the change record.

Deliverable. Scheduled comparison pipeline with Terraform drift check and runbook.

Common pitfalls.

- Treating exit code 2 from -detailed-exitcode as a failure [9].
- Expecting cron to run exactly on time and in local time zone [25].
- Running terraform apply from the comparison workflow.

GenNoorTech
For more info - https://gennoor.com

- No alert when the scheduled run itself fails.

Knowledge check.

1. What does terraform plan -detailed-exitcode return when changes are present? Answer: 2 [9].
2. In what time zone do scheduled workflows run by default? Answer: UTC [25].
3. How does Terraform JSON represent a replacement? Answer: change.actions of ['delete','create'] or ['create','delete'] [24].
4. What is the shortest schedule interval in GitHub Actions? Answer: Once every 5 minutes [25].

Slides and assets to prepare.

- Workflow file starter.
- Lab Terraform project for the cloud load balancer.
- Plan-to-canonical converter.
- Runbook template.

Sources for this module. 9, 24, 25, 27

Module 10: Capstone: end-to-end physical-to-cloud comparison

Goal. Teams run the full pipeline on the lab physical F5 and cloud NetScaler and present an approval package for acceptance against the module 1 checklist.

Outline topics.

- Collect, normalise, plan, diff and explain a physical F5 to cloud NetScaler migration end to end
- Produce the approval package: plan, severity report, Claude explanation, evidence, ServiceNow change, pull request
- Demo and acceptance review against the module one checklist

Learning objectives.

- Learners can run collect, normalize, plan, diff and explain end to end.
- Learners can assemble the approval package with change record and pull request.
- Learners can defend the package against the module 1 checklist.

Run the pipeline

- Collect from the lab F5 [15] and cloud NetScaler [16].
- Normalize, plan, diff and explain with versions recorded.
- Seed one surprise mismatch chosen by the instructor.
- Teams may not edit fixtures or severity rules after the surprise mismatch is seeded.

The approval package

- Plan, severity report, Claude explanation, evidence index, unknowns and manual items.
- ServiceNow change created via the Table API [27].

- Pull request under branch protection [26].
- Unknowns and manual items (for example iRule logic and named expressions [3]) are listed with owners.

Acceptance review

- Check every module 1 checklist row.
- Confirm no change was applied and the attempt log is clean.
- Limitations statement and next steps.
- Reviewers replay one diff line from report to raw evidence to show traceability.

Instructor demo.

1. Seed the surprise mismatch in the target.
2. Show the acceptance checklist and scoring rubric.
3. Model a two-minute acceptance walkthrough.
4. Show how the audit log will be checked.

Hands-on lab. Run the full pipeline on the lab physical F5 and cloud NetScaler, produce the approval package with ServiceNow change and pull request, and present it for acceptance review against the module one checklist.

1. Run collectors on both platforms.
2. Normalize and review unknowns.
3. Generate and review the plan.
4. Run the diff engine and confirm the surprise mismatch is found.
5. Generate explanations and remediation proposals.
6. Create the change record and open the pull request.
7. Assemble the approval package.
8. Present for acceptance review.

Lab success criteria.

- The surprise mismatch is found and classified correctly.
- Every package claim traces to evidence.
- No change applied to any platform.
- All module 1 checklist rows addressed.

Deliverable. Complete comparison pipeline and approval package.

Common pitfalls.

- Skipping the unknowns review under time pressure.
- Presenting Claude's explanation without the report it cites.
- Leaving manual items (iRule logic, named expressions) out of the package.

Knowledge check.

GenMor Tech
INNOVATE. BUILD.
For more info - https://genmor.com

1. What proves the package is complete? Answer: Every module 1 checklist row is addressed with evidence.
2. What must the audit show? Answer: Every prompt, tool call and decision, and no applied change.
3. Who approves the comparison? Answer: The named approver on the change record and reviewers on the pull request.

Slides and assets to prepare.

- Capstone brief.
- Acceptance rubric.
- Approval package template.

Sources for this module. 3, 15, 16, 26, 27

6. Capstone plan

A repeatable pipeline that collects configuration from two platforms, normalises both into a canonical model, has Claude draft the migration plan and explain a deterministic diff, and delivers an approval package through a GitHub pull request and a ServiceNow change record with a full audit trail and no change applied without approval.

Teams run the complete pipeline on the lab physical F5 and cloud NetScaler: collect, normalize, plan, diff and explain, then deliver an approval package with a ServiceNow change and a GitHub pull request, with no change applied and a full audit trail.

Milestones

1. Collection complete with raw evidence for both platforms.
2. Canonical models and unknowns reviewed.
3. Plan generated, reviewed and committed.
4. Diff report produced; instructor's surprise mismatch found.
5. Explanations and remediation proposals validated against the report.
6. Change record and pull request created; package presented.

Evaluation notes

- Surprise mismatch found and correctly classified (pass/fail).
- Traceability: every explanation line cites an existing report line.
- Completeness of unknowns and manual items.
- No write action attempted outside the approval gate; attempts logged.
- Clarity of the limitations statement.

Assessment weights

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

7. Promotional video: research behind the script

Opening scenario. A cut-over on a fixed date with a target configuration rebuilt by hand from a spreadsheet is a teaching scenario, but the underlying risk is real: vendor tools compare or migrate within one ecosystem [2][3][6][7], and defaults differ across platforms, for example F5's 180-second source-address persistence [18] versus NetScaler's lbvserver persistence timeout default of 2 [16].

GenNoorTech TRAIN · INNOVATE · BUILD For more info - https://gennoor.com

Claim or on-screen text	Basis
10 build modules	Matches the outline in content.py.
3 migration directions	Taken directly from the client input (cloud-cloud, physical to cloud, cloud to physical).
5 platform API families	Course design: F5 iControl REST, NetScaler NITRO, Radware Alteon, Cisco APIC and Terraform state [15][16][22][23][24]. Alteon is CLI-export based in the lab, so 'API family' is loose wording for Radware.
1 canonical schema for every object	Course design using Pydantic models [28].
0 changes applied without approval	Design goal enforced by read-only tools, branch protection and change approval [14][26][27]; a property of the course build.
'f5: iControl REST /mgmt/tm/ltm … netscaler: NITRO /nitro/v1/config'	Real endpoint families [15][16]; counts are illustrative.
'6 unknowns (f5 fastL4 profile: no NITRO equivalent)'	Fast L4 is a real F5 profile [20]. That it has 'no NITRO equivalent' is a simplification for the lab narrative (NetScaler has its own TCP profiles); present it as an unmapped field requiring review, not as a vendor fact.
'gap: F5 iRule redirect_legacy has no NITRO equivalent → responder policy'	Responder policies handle redirects on NetScaler [21]; the specific iRule is a teaching scenario.
'risk: 2 pools use priority groups; NetScaler backup vserver proposed'	F5 priority groups [19] and NetScaler backup virtual servers [17] both exist; the mapping is a design choice, not a one-to-one equivalent.
'unknown: persistence timeout on vs_app_api (not in inventory)'	Teaching scenario showing the planner must mark missing data unknown.
'confidence: 0.82 owner: netops-lead cited: inventory#f5-0007'	Illustrative output of the course plan schema; values invented.
Diff line: 'vs_app_api persistence: source_ip 180s BLOCKING / source value 300s' and 'session drops likely after 180s idle; set timeout 300'	Teaching scenario. 180 s matches F5's source-addr default [18]; on the NetScaler target the equivalent setting is the lbvserver timeout (default 2) [16][17], so the lab should show the value after unit normalization. Wording 'session drops' should be read as 'persistence lost', not dropped connections.
'monitor interval: 10s (source 5s) WARNING'	Teaching scenario; interval values are illustrative and show a warning-class difference.
'servicenow: CHG0041877 created via Table API approver: netops-lead'	Change records are created with POST on the Table API [27] and routed by Flow Designer [31]; number invented.
'github: PR #58 opened checks: diff-engine passed, drift-check passed'	Status checks and required reviews via branch protection [26]; illustrative.

Terraform plan output compared against live state before apply	terraform plan and its JSON output support this [9][24].
Scheduled drift checks between platforms	GitHub Actions schedule event [25].

References

[1] ltm virtual - F5 TMSH Reference v17. F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/ltm/ltm_virtual.html
Supports: Comparison discipline; collection; canonical model. A BIG-IP virtual server is defined by properties such as destination (default any:any), pool, profiles, persist, rules (iRules), policies, vlans and source-address-translation; the snat and snatpool attributes are deprecated in favor of source-address-translation; show virtual displays statistics and status.

[2] System base operations. NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/system/basic-operations.html
Supports: Comparison discipline; collection. NetScaler configuration is stored in /nsconfig/ns.conf and must be saved with save ns config to survive restarts; show ns runningConfig and show ns ns.conf display running and saved configuration; diff ns config <configfile> <configfile2> (GUI: System > Diagnostics > Configuration difference) compares two configuration files.

[3] Migrate NetScaler application configuration using StyleBooks Configuration Builder. NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/netscaler-application-delivery-management-software/current-release/stylebooks/migrate-citirx-adc-application-configuration-using-stylebooks
Supports: Migration planning with Claude. The Configuration Builder takes a NetScaler instance, pasted ns.conf or application CLI commands, or an uploaded file, produces a StyleBook and config pack for duplicating configuration to other instances, reports missed or unsupported configuration to apply separately, and does not identify named expressions and responder HTML pages, which must be configured on the target before migration.

[4] What is an Application Load Balancer?. Amazon Web Services.
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
Supports: Comparison discipline; migration planning (cloud targets). An ALB has listeners (protocol and port), rules (priority, conditions, actions) with a mandatory default rule per listener, and target groups with per-target-group health checks; a target can be registered with multiple target groups; the default routing algorithm is round robin, with least outstanding requests as an alternative.

[5] Application gateway components. Microsoft Learn. https://learn.microsoft.com/en-us/azure/application-gateway/application-gateway-components
Supports: Comparison discipline; migration planning (cloud targets). Azure Application Gateway uses frontend IPs, listeners (multi-site listeners support wildcards and up to five host names), request routing rules (basic, advanced or path-based; one listener per rule), backend settings, backend pools and health probes; HTTP/2 applies to client connections only and backend communication is always HTTP/1.1.

[6] F5 BIG-IP Automation Config Converter (BIG-IP ACC). F5 (clouddocs.f5.com).
https://clouddocs.f5.com/products/extensions/f5-automation-config-converter/latest/
Supports: Migration planning with Claude. BIG-IP ACC (1.24) converts BIG-IP configuration files into AS3 or Declarative Onboarding declarations based on supported classes; it is a community-supported offering with source code and issues on GitHub.

[7] Welcome to the F5 BIG-IP Migration Assistant - Now the F5 Journeys App. F5 DevCentral community.
https://community.f5.com/t/welcome-to-the-f5-big-ip-migration-assistant-now-the-f5-journeys-app/65567
Supports: Migration planning with Claude. The BIG-IP Migration Assistant is deprecated and replaced by F5

Journeys (github.com/f5devcentral/f5-journeys), which loads UCS or UCS plus AS3 sources, migrates from any version starting at 11.5.0 to a higher one including VELOS and rSeries, flags feature-parity gaps with built-in fixes, and can migrate applications to AS3.

[8] Tool use with Claude. Anthropic (platform.claude.com). https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview
Supports: Claude explains the differences. To have Claude call a function you define, pass a tool with an input_schema; Claude responds with stop_reason tool_use and tool_use blocks, your code runs the operation and returns a tool_result; adding strict: true to a custom tool definition ensures Claude's tool calls always match the schema.

[9] terraform plan command reference. HashiCorp Developer.
https://developer.hashicorp.com/terraform/cli/commands/plan
Supports: Scheduling, Terraform plan checks and GitHub Actions. terraform plan compares configuration with prior state and shows + create, - destroy, ~ update in-place and -/+ replace; -detailed-exitcode returns 0 (no changes), 1 (error) or 2 (changes present); -out=FILE saves a plan for later apply, otherwise the plan is speculative; -refresh-only creates a plan that only updates state to match changes made outside Terraform.

[10] How Claude remembers your project. Anthropic (Claude Code docs).
https://code.claude.com/docs/en/memory
Supports: Setting up Claude Code, the Agent SDK and MCP. Project instructions live in ./CLAUDE.md or ./.claude/CLAUDE.md and are shared via version control; /init generates a starting file; @path imports pull in other files; CLAUDE.md is context, not enforced configuration, so blocking actions requires a PreToolUse hook; files should stay under about 200 lines.

[11] Connect Claude Code to tools via MCP. Anthropic (Claude Code docs).
https://code.claude.com/docs/en/mcp
Supports: Setting up Claude Code, the Agent SDK and MCP. claude mcp add registers HTTP (recommended for remote) or stdio servers (arguments after -- go to the process); SSE is deprecated; scopes are local (default), project (.mcp.json, shared via Git) and user; .mcp.json expands ${VAR} and ${VAR:-default} so credentials stay in the environment.

[12] Give Claude custom tools (Agent SDK). Anthropic (Claude Code docs).
https://code.claude.com/docs/en/agent-sdk/custom-tools
Supports: Setting up the Agent SDK; Claude explains the differences. Python tools are defined with @tool and wrapped with create_sdk_mcp_server, passed via ClaudeAgentOptions(mcp_servers=...), and named mcp__{server}__{tool} in allowed_tools; the tools option removes built-ins from Claude's context; handlers return content and may set is_error; readOnlyHint is metadata, not enforcement.

[13] Get structured output from agents. Anthropic (Claude Code docs). https://code.claude.com/docs/en/agent-sdk/structured-outputs
Supports: Migration planning with Claude; Claude explains the differences. output_format={'type': 'json_schema', 'schema': ...} returns validated JSON in ResultMessage.structured_output; Pydantic's model_json_schema() can supply the schema; validation uses JSON Schema draft-07 with re-prompting, and failure yields subtype error_max_structured_output_retries.

[14] Tools (MCP specification 2025-06-18). Model Context Protocol.
https://modelcontextprotocol.io/specification/2025-06-18/server/tools
Supports: Setting up MCP; guardrails. MCP tools are listed with tools/list and invoked with tools/call; definitions include inputSchema and optional outputSchema and annotations; results can carry structuredContent and isError; there should always be a human in the loop able to deny invocations, annotations are untrusted unless from trusted servers, and clients should log tool usage for audit.

[15] APIRef_tm_ltm_virtual (iControl REST). F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_ltm_virtual.html
Supports: Collecting configuration from every platform. The iControl REST collection URI for virtual servers is /mgmt/tm/ltm/virtual (OPTIONS, GET) and the resource URI is /mgmt/tm/ltm/virtual/~resource id (OPTIONS, GET, PUT, PATCH, DELETE, POST), with natural key name, partition and subPath.

[16] lbvserver configuration (NITRO API 13.1). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/13-1/configuration/lb/lbvserver
Supports: Collecting configuration; normalizing. GET /nitro/v1/config/lbvserver lists virtual servers (with attrs, filter, view=summary and pagesize/pageno), /nitro/v1/config/lbvserver/<name> returns one and ?count=yes returns a count; requests use Cookie NITRO_AUTH_TOKEN or X-NITRO-USER/X-NITRO-PASS; fields include persistencetype (SOURCEIP, COOKIEINSERT, SSLSESSION, RULE, NONE and others), timeout (default 2, range 0 to 1440), lbmethod (default LEASTCONNECTION), and read-only curstate and effectivestate.

[17] lb vserver - ADC CLI Commands. NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-command-reference-int/current-release/lb/lb-vserver
Supports: Normalizing; migration planning. In the CLI reference, timeout is the time period for which a persistence session is in effect (default 2, maximum 1440); persistenceBackup and backupPersistenceTimeout provide backup persistence; -backupVServer names a backup virtual server; show lb vserver displays statistics for a virtual server.

[18] ltm persistence source-addr - F5 TMSH Reference. F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/ltm/ltm_persistence_source-addr.html
Supports: Normalizing; the deterministic diff engine. The source-addr persistence profile timeout defaults to 180 seconds; mask defaults to ::; match-across-pools, match-across-services and match-across-virtuals default to disabled.

[19] ltm pool - F5 TMSH Reference. F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/ltm/ltm_pool.html
Supports: Normalizing; migration planning. Pools default to round-robin load balancing; members can be placed in priority-group 0 to 65535 with traffic sent to groups in priority order; min-active-members (default 0) controls priority-based activation; member ratio defaults to 1; member monitor defaults to the pool's monitors.

[20] ltm profile fastl4 - F5 TMSH Reference. F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/ltm/ltm_profile_fastl4.html
Supports: Normalizing; promo claims check. The Fast L4 profile is the default profile the system uses for a basic configuration of non-UDP traffic, with options such as idle-timeout; changes to an active Fast L4 profile take effect for new connections after the idle-timeout passes.

[21] Responder. NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/appexpert/responder.html
Supports: Migration planning with Claude. A NetScaler responder action can generate a custom response, redirect a request or reset a connection; a responder policy selects the traffic and is bound to a bind point such as a load balancing virtual server; responder works on client-side parameters, unlike rewrite.

[22] Cisco APIC REST API Configuration Guide - Using the REST API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-rest-api-configuration-guide/cisco-apic-rest-api-configuration-guide/m_using_the_rest_api.html
Supports: Collecting configuration from every platform. APIC objects are read at /api/mo/{dn}.{json|xml} and classes at /api/class/{className}; aaaLogin returns a token used as APIC-cookie (default session timeout 600 s); query-target takes self, children or subtree; query-target-filter supports operators such as eq and wcard; page and page-size paginate; GET makes no changes.

[23] Alteon 31.0.11.0 Release Notes - Maintenance fixes. Radware. https://portals.radware.com/releasenotes/Alteon_Release_Notes_31.0.11.0/Alteon_Release_Notes/AlteonOS-31_0_11_0-RN.1.41.html
Supports: Collecting configuration from every platform. Radware's fix list references Alteon CLI paths such as /cfg/slb/virt <vsid>/service <vport> for virtual service configuration, the diff flash command that shows pending versus saved configuration, and /stats/slb/virt and /stats/slb/group for statistics.

[24] JSON Output Format. HashiCorp Developer. https://developer.hashicorp.com/terraform/internals/json-format
Supports: Collecting configuration (Terraform state); Scheduling, Terraform plan checks and GitHub Actions. terraform show -json <FILE> renders a plan or state file as JSON with a format_version key ('1.0' as of Terraform

1.1.0); plans contain resource_changes (each with address and change.actions such as ['no-op'], ['create'], ['update'], ['delete','create'] or ['delete'], plus before and after values) and resource_drift for changes detected outside Terraform.

[25] Events that trigger workflows. GitHub Docs. https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows
Supports: Scheduling, Terraform plan checks and GitHub Actions. The schedule event uses POSIX cron syntax, runs in UTC by default (an IANA timezone can be set), runs on the latest commit on the default branch, and the shortest interval is once every 5 minutes; multiple schedules can be combined and the triggering one read from github.event.schedule.

[26] About protected branches. GitHub Docs. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
Supports: Guardrails, approvals and audit. Branch protection rules can require pull request reviews before merging (a set number of approving reviews, optionally from code owners), require status checks, and dismiss stale approvals when new commits are pushed; by default force pushes and deletions are blocked on protected branches.

[27] Table API. ServiceNow Product Documentation. https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/inbound-rest/concept/c_TableAPI.html
Supports: Guardrails, approvals and audit; scheduling. POST /api/now/table/{tableName} inserts one record (no bulk insert) and PATCH/PUT /api/now/table/{tableName}/{sys_id} update a record; GET supports sysparm_query, sysparm_fields, sysparm_limit (default 10000), sysparm_offset and sysparm_display_value.

[28] Models (Pydantic). Pydantic. https://docs.pydantic.dev/latest/concepts/models/
Supports: Normalising into a canonical configuration model. Pydantic models validate input on instantiation or with model_validate() and model_validate_json(), raising ValidationError when data cannot be parsed; model_dump() and model_dump_json() serialize; the extra setting is 'ignore' by default, 'forbid' rejects unknown keys and 'allow' keeps them in __pydantic_extra__.

[29] Try BIG-IP and BIG-IQ Virtual Editions. F5. https://www.f5.com/trials/big-ip-virtual-edition
Supports: Lab environment. F5 offers a 30-day free trial of BIG-IP Virtual Edition requested through MyF5 (My Products & Plans > Trials > BIG-IP Virtual Edition & BIG-IQ Virtual Edition).

[30] Licensing (NetScaler VPX Express). NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/licensing.html
Supports: Lab environment. NetScaler VPX Express for on-premises and cloud deployments needs no license file and offers 20 Mbps bandwidth, up to 250 SSL sessions and 20 Mbps SSL throughput; from 14.1 build 29.65 it includes the advanced Premium features.

[31] Architecture Overview (Flow Designer / Workflow Studio). ServiceNow Product Documentation. https://www.servicenow.com/docs/bundle/washingtondc-build-workflows/page/administer/flow-designer/concept/flow-designer-arch-overview.html
Supports: Guardrails, approvals and audit. Flows start from record-based, schedule-based or application-based triggers; queued events are processed in the order added, so a flow may start with a delay; process plans are compiled just in time and cached when unchanged.

For more info - https://gennoor.com